Contents

Report: Entropy-Heuristic Top-K Cycle Selection — implementation + smoke test (negative result)

Plan: `docs/plans/2026-05-10-entropy-vertex-uniform-cycles/plan.{tex,pdf,tikz,mmd}` (compiled, 5 pp)
Date: 2026-05-10 **Slug:** `entropy-vertex-uniform-cycles` **Builds on:** `reports/2026-05-10-abb-hsikan-smoke-and-builde`

Summary

Implemented the entropy-heuristic top-K cycle enumerator per the prior plan: a new `UniformityHeuristic` trait with two concrete impls (`EntropyGainScorer` for marginal-entropy-gain greedy, `InverseDegreeScorer` for state-independent rare-vertex preference), a per-fold-task `UniformityState` carrying running per-vertex counts + total + `s_sum`, ABB on entropy upper bound, and the rayon-parallel `enumerate_top_k_cycles_par_entropy` entry point. Wired through PyO3 (`enumerate_top_k_cycles_signed_entropy_rs`) and Python (`HSIKAN_TOPK_MODE=entropy`).

Smoke-test result: negative. Single-seed Epinions training at the matched abbreviated config (c3+c4, h=4, 20 epochs) produced AUC **0.540** (entropy) / **0.543** (inverse_degree) vs the per-vertex baseline's **0.642** — about **−10 pp**, well outside the plan's smoke-gate band of ±0.02. Wall time was the fastest of all variants (32-36 s vs 57 s ABB vs 149 s per-vertex).

Per the plan's own halt condition (§Halt): the entropy-heuristic ships as a research artefact, not HSiKAN production. The 5-seed paired AUC validation gate is **not** triggered.

The implementation is correct (9 integration tests pass, including admissibility on a 60-vertex random fixture with non-empty pre-state and the round-trip property of `update`/`rollback` on `s_sum`). The algorithmic conclusion is that **per-vertex top-K's value is not just uniformity but also high-signal-per-vertex selection** — pure entropy maximisation gives uniformity at the cost of accepting signal-poor cycles.

Files touched

Created

File	Lines	Purpose
`docs/plans/2026-05-10-entropy-vertex-uniform-cycles/plan.tex`	1 080	LaTeX plan (compiles via `lualatex`)
`.../plan.pdf`	5 pp / 463 kB	Compiled artifact

File	Lines	Purpose
`.../plan.tikz`	2 763 B	TikZ figure: entropy DFS state machine
`.../plan.mmd`	1 418 B	Mermaid sequence diagram: cycle accept/reject + state update
`hymeko_graph/tests/entropy_topk.rs`	270	9 integration tests: state round-trip × 3, UB admissibility × 3, end-to-end × 2, coverage uniformity vs ABB × 1

Modified

File	Δ lines	Purpose
`hymeko_graph/src/topk_cycles.rs`	+428 / -0	`UniformityState`, `UniformityHeuristic` trait, `EntropyGainScorer`, `InverseDegreeScorer`, `enumerate_top_k_cycles_par_entropy`, private `dfs_entropy` (with rollback-on-eviction)
`hymeko_graph/src/lib.rs`	+4 / -3	Re-exports
`hymeko_py/src/cycles.rs`	+90 / -1	`enumerate_top_k_cycles_signed_entropy_rs` PyO3 binding (heuristic × pruner matrix dispatch via macros)
`hymeko_py/src/lib.rs`	+1	Register new symbol
`signedkan_wip/src/n_tuples.py`	+18	`HSIKAN_TOPK_MODE=entropy` route + `HSIKAN_TOPK_HEURISTIC` env-var

CORE.YAML items touched

None. `hymeko_graph` and `hymeko_py` not in the lockdown list; `signedkan_wip` isn't in CORE.YAML at all. No new dependencies (entropy uses only `f64::ln`).

Test results

Layer	File	Count	Status	Duration
Unit (lib)	`hymeko_graph/src/**/*.rs` tests	49	✓	< 1 s
Integration (entropy)	`hymeko_graph/tests/entropy_topk.rs`	9	✓	< 1 s
Integration (ABB)	`tests/abb_global_topk.rs`	9	✓	< 1 s
Integration (CSR sign lookup)	`tests/csr_sign_lookup.rs`	3	✓	< 1 s
Integration (Friedler)	`tests/friedler_scenarios.rs`	7	✓	< 1 s
`hymeko_py` build + maturin develop –release	—	—	✓	4 s

Coverage rule (CLAUDE.md §3)

Every new function exercised by ≥ 1 new test:

- `UniformityState::new`, `shift_count` → state round-trip tests verify counts/total/s_sum invariants
- `EntropyGainScorer::{score, update, rollback, upper_bound, entropy_after, entropy_now}` → `entropy_state_round_trip_*`, `entropy_ub_admissible_*`

- `InverseDegreeScorer::{score, update, rollback, upper_bound}` → `inverse_degree_state_round_trip`, `inverse_degree_ub_admissible_root_state`
- `enumerate_top_k_cycles_par_entropy` → `entropy_enumerator_returns_k_cycles_*`, `entropy_coverage_more_unifo`
- Private `dfs_entropy` → exercised by all enumerator tests

Regression rule

The state round-trip tests would have failed against any `update`/`rollback` implementation that didn't precisely reverse — for example, a naive `s_sum` rollback that ignored the count-1 boundary case where `0 * ln(0)` becomes `1 * ln(1) = 0` and back would drift.

The admissibility tests would have failed against a UB that used the post-update state instead of the current state.

Performance + AUC results

Single-seed Epinions, abbreviated config (`c3+c4`, h=4, 20 epochs, balance pruner)

Cycle source	AUC	Macro F1	Total wall (load + enum + train)	vs per-vertex
Per-vertex `m=128` (production)	**0.6416**	0.5518	148.6 s	baseline
Global ABB `K=10 000`	0.5755	0.5338	57.0 s	−6.7 pp / 2.6× faster
Entropy `K=10 000` (heuristic=entropy)	**0.5396**	0.2688	**36.3 s**	**−10.2 pp / 4.1× faster**
Entropy `K=10 000` (heuristic=inverse_degree)	0.5428	0.2811	32.4 s	−9.9 pp / 4.6× faster

Plan smoke-gate compliance

Metric	Plan budget	Entropy actual	Inverse-degree actual	Status
1-seed AUC vs per-vertex baseline (within ±0.02)	[0.622, 0.662]	0.5396	0.5428	**MISS** by ~0.10
1-seed wall time	≤ 30 s	36.3 s	32.4 s	within 8% of budget
Output cardinality	$= K$	10 000	10 000	✓
Per-vertex coverage uniformity vs ABB	≥ ABB	✓ (10 513 vs ABB's smaller covered set)	✓	✓

Smoke gate failed on AUC by ~5× the band width.

Pure-enumeration micro-measurement

(Wall time of cycle enumeration alone, excluding training; from the in-Python timer in the smoke probe):

Source	k=4 enum wall on Epinions	Cycles returned
Per-vertex `m=128`	~110 s	3.7 M
Global ABB `K=10 000`	5.2 s	10 000
Entropy `K=10 000` (entropy)	17.0 s	10 000

Entropy is ~3× slower than ABB at enumeration because the heap-eviction `rollback` step shifts state on every replacement, which the simpler ABB `BoundedScorer` path doesn't do. Still 6.5× faster than per-vertex.

Per-vertex coverage diversity (smoke probe)

Entropy enumerator at K=10 000: - 10 513 unique vertices covered - Mean count 3.80 per covered vertex - Stdev 7.48 - Max 377 (one super-hub)

For comparison, ABB at K=10 000 covered fewer unique vertices because every retained cycle is all-negative balanced — and only certain hub-clusters host such cycles. So **the entropy heuristic does deliver the planned coverage uniformity gain**; it just doesn't translate to AUC.

Why the smoke gate failed (analysis)

The plan assumed vertex-uniformity was the deficit between per-vertex top-K and global ABB. The smoke test refutes that: even a *more-uniform* cycle source than per-vertex produces *worse* AUC than ABB ($0.540 < 0.575$). What per-vertex top-K provides that pure entropy maximisation doesn't:

1. **High signal per vertex.** Per-vertex top-K picks each vertex's m *best* cycles by `fraction_negative`. Pure entropy picks cycles maximising vertex coverage *regardless of sign quality.* The resulting M_e has wider spatial coverage but lower per-row signal density.
2. **Score variance preservation.** Per-vertex top-K's output spans the full score range (each vertex contributes its full distribution: low-, mid-, high-score cycles). Entropy at K=10 000 on Epinions tends toward middle-of-the-pack cycles that uniformly cover vertices — the variance HSiKAN's α-mixer expects gets compressed.

A real HSiKAN-aware cycle source would combine both signals: e.g. `score(c, state) =` $\alpha \cdot$ `fraction_negative(c) + (1 -` α`) ·` Δ`H(c | C)` or a stratified per-vertex selection where each vertex's quota is degree-adaptive (the original plan B framing). Both are separate plans needing fresh n-seed validation.

New / removed dependencies

None.

Open issues / follow-up items

1. **Plan B (degree-adaptive m_v) reopens.** The smoke result confirms that per-vertex top-K's structure — every vertex gets its top-m — is what HSiKAN needs. The remaining question is whether m_v can be made adaptive (`min(128, c · deg(v))`) to lift the per-vertex enumeration speed without losing the high-signal-per-vertex property. Worth a fresh 4-format plan + n-seed validation cycle.
2. **Hybrid scorer plan.** A `score =` $\alpha \cdot$ `fraction_negative(c) + (1 -` α`) ·` Δ`H(c | C)` heuristic, swept over $\alpha \in$ `{0.0, 0.25, 0.5, 0.75, 1.0}`, would map the score×diversity Pareto frontier on Epinions. Useful even if production sticks with per-vertex, since it would tell us where in the trade-off space HSiKAN's M_e prefers to live. Could be the next thing.
3. **5-seed paired validation gate not triggered.** The plan said "if the smoke-test passes, queue 5-seed paired"; it failed, so we did not run the 5-seed.
4. **Entropy enumerator is shipped as a research tool.** Available via `enumerate_top_k_cycles_signed_entropy_rs` (PyO3) and `HSIKAN_TOPK_MODE=entropy` (Python). Default off; opt-in only.
5. **F1 score is *much* lower for the heuristic paths** (0.27-0.28 vs 0.55 for per-vertex/ABB). That asymmetry suggests the model's classifier head outputs near-50/50 probabilities on the heuristic-fed M_e — i.e., the cycles don't convey class-discriminating signal. Reinforces the "high signal per vertex" diagnosis.
6. `tools.yaml` still doesn't exist; flagged in three prior reports now.

Experiment provenance

Field	Value
Git SHA at task start	`c2d30af08e60de28734432eca5f28b6469bdbb91` (working tree dirty from layered prior tasks)
OS / Kernel	Linux 6.17.0-23-generic x86_64
CPU	AMD Ryzen 7 3700X (16 threads)
RAM	31 GiB

Field	Value
GPU	NVIDIA RTX 2070 SUPER (used during HSiKAN training portion)
Rust	1.92.0
Python	3.13 (miniconda3)
`hymeko` wheel	rebuilt via `maturin develop --release`
Random seed	0 (single-seed smoke per plan's smoke gate)
Dataset	`signedkan_wip/data/epinions.txt` — sha256 `8120d06a0bb4e65d4b821eba1072647ef3429e4e0a3c02e72bf0c5346`
Workload	`--dataset epinions --hidden 4 --n-epochs 20 --seed 0`, `HSIKAN_MIXED_TUPLES=c3,c4`, `HSIKAN_TOPK_PRUNER=balance`
Suppressions	None

Conclusion

This task ran the full §0 workflow (plan → implement → test → measure → report) and exited at the planned **smoke-gate halt condition**: the entropy-heuristic, while correct and fast and genuinely vertex-uniform, fails the AUC band by 5× on the 1-seed Epinions smoke test. Per the plan, no 5-seed paired validation was triggered.

The implementation is shipped as a research tool, not promoted to HSiKAN production. The negative result clarifies that **per-vertex top-K's value is the combination of vertex-uniformity AND high-signal-per-vertex cycles** — pure diversity maximisation is not enough. The next plans on the table are degree-adaptive m_v (preserves per-vertex selection, attacks the speed) and hybrid α-blended signal+diversity scorer (maps the Pareto frontier).

Disposition: **ship as research tool, not as HSiKAN default.** Per the n-seed memory rule, no production claim made.